Reporte: 2p ejercicio 5 lista

En este programa implementamos una Estructura de Datos de tipo Lista Estática utilizando un arreglo de tamaño fijo bajo el paradigma de Programación Orientada a Objetos. La clase ListaEstatica encapsula el arreglo de datos, la capacidad máxima y la variable de control de tamaño, ofreciendo métodos para insertar, eliminar, buscar y mostrar los elementos.

Código Completo:

```
#include <iostream>
using namespace std;

class ListaEstatica {
private:
    static const int CAPACIDAD = 10; // Tamaño máximo fijo del arreglo
    int datos[CAPACIDAD];
    int tamano; // Cantidad actual de elementos

public:
    // Constructor
    ListaEstatica() {
        tamano = 0;
    }

    // Verificar si la lista está llena
    bool estaLlena() const {
        return tamano == CAPACIDAD;
    }

    // Verificar si la lista está vacía
    bool estaVacia() const {
        return tamano == 0;
    }

    // Insertar un elemento al final de la lista
    void insertar(int valor) {
        if (estaLlena()) {
            cout << "Error: La lista esta llena. No se puede insertar " << valor <<
".\n";
            return;
        }
        datos[tamano] = valor;
        tamano++;
        cout << "Elemento " << valor << " insertado correctamente.\n";
    }

    // Insertar en una posición específica (0-indexed)
    void insertarEn(int pos, int valor) {
        if (estaLlena()) {
            cout << "Error: La lista esta llena.\n";
            return;
        }
        if (pos < 0 || pos > tamano) {
            cout << "Error: Posicion invalida.\n";
            return;
```

```
        }
        for (int i = tamano; i > pos; i--) {
            datos[i] = datos[i - 1];
        }
        datos[pos] = valor;
        tamano++;
        cout << "Elemento " << valor << " insertado en la posicion " << pos << ".\n";
    }

    // Eliminar por valor
    void eliminar(int valor) {
        if (estaVacia()) {
            cout << "Error: La lista esta vacia.\n";
            return;
        }
        int pos = -1;
        for (int i = 0; i < tamano; i++) {
            if (datos[i] == valor) {
                pos = i;
                break;
            }
        }
        if (pos == -1) {
            cout << "Error: El elemento " << valor << " no se encuentra en la lista.\n";
            return;
        }
        for (int i = pos; i < tamano - 1; i++) {
            datos[i] = datos[i + 1];
        }
        tamano--;
        cout << "Elemento " << valor << " eliminado de la lista.\n";
    }

    // Buscar la posición de un elemento
    int buscar(int valor) const {
        for (int i = 0; i < tamano; i++) {
            if (datos[i] == valor) {
                return i;
            }
        }
        return -1;
    }

    // Imprimir el contenido de la lista
    void mostrar() const {
        if (estaVacia()) {
            cout << "La lista esta vacia.\n";
            return;
        }
        cout << "Lista [" << tamano << "/" << CAPACIDAD << "]: [ ";
        for (int i = 0; i < tamano; i++) {
            cout << datos[i] << " ";
        }
        cout << "]\n";
    }
};

int main() {
```

```
    ListaEstatica lista;
    int opcion, valor, pos;

    do {
        cout << "\n======================================\n";
        cout << "  LISTA ESTATICA (CON ARREGLO) - POO  \n";
        cout << "======================================\n";
        cout << "1. Insertar al final\n";
        cout << "2. Insertar en posicion específica\n";
        cout << "3. Eliminar por valor\n";
        cout << "4. Buscar elemento\n";
        cout << "5. Mostrar lista\n";
        cout << "6. Salir\n";
        cout << "Selecciona una opcion: ";
        cin >> opcion;

        switch (opcion) {
            case 1:
                cout << "Ingresa el valor a insertar: ";
                cin >> valor;
                lista.insertar(valor);
                break;
            case 2:
                cout << "Ingresa la posicion: ";
                cin >> pos;
                cout << "Ingresa el valor a insertar: ";
                cin >> valor;
                lista.insertarEn(pos, valor);
                break;
            case 3:
                cout << "Ingresa el valor a eliminar: ";
                cin >> valor;
                lista.eliminar(valor);
                break;
            case 4:
                cout << "Ingresa el valor a buscar: ";
                cin >> valor;
                pos = lista.buscar(valor);
                if (pos != -1) {
                    cout << "El elemento " << valor << " se encuentra en el indice " <<
pos << ".\n";
                } else {
                    cout << "El elemento " << valor << " no esta en la lista.\n";
                }
                break;
            case 5:
                lista.mostrar();
                break;
            case 6:
                cout << "Saliendo del programa...\n";
                break;
            default:
                cout << "Opcion invalida. Intenta de nuevo.\n";
        }
    } while (opcion != 6);
```

```
    return 0;
}
```

Explicación del Código:

1. Encapsulamiento y Atributos Privados (private:)
Se declara la constante estática CAPACIDAD (fijada en 10), el arreglo unidimensional datos[CAPACIDAD] y el contador tamano dentro de la sección privada de la clase ListaEstatica. Esto evita la manipulación directa de la memoria física del arreglo desde el exterior.

2. Métodos de Control de Estado (estaLlena() y estaVacia())
Validan los límites operativos de la estructura de datos: estaLlena() verifica si tamano llegó a la CAPACIDAD máxima, mientras que estaVacia() valida si tamano es 0. Ambos son métodos const ya que solo consultan información.

3. Operaciones de Inserción (insertar() e insertarEn())
insertar() agrega el elemento en la posición inmediata disponible y aumenta tamano. Por su parte, insertarEn(pos, valor) desplaza los elementos existentes una posición hacia la derecha desde el final hasta la posición deseada antes de guardar el nuevo valor.

4. Operaciones de Búsqueda y Eliminación (eliminar() y buscar())
buscar() realiza un recorrido secuencial sobre el arreglo retornando el índice del valor o -1 si no existe. eliminar() localiza la primera coincidencia y desplaza los elementos a la izquierda para cubrir la posición vacía, decrementando tamano.

5. Menú Interactivo en la Función Principal (main)
Instancia el objeto 'lista' e implementa un menú interactivo mediante un bucle do-while con una estructura switch-case para permitir al usuario probar dinámicamente cada una de las operaciones de la lista estática.